

62. Unique Paths Medium

Acceptance: 66.5% | [LeetCode](#)

There is a robot on an $m \times n$ grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers `m` and `n`, return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:

```
**Input:** m = 3, n = 7
**Output:** 28
```

Example 2:

```
**Input:** m = 3, n = 2
**Output:** 3
**Explanation:** From the top-left corner, there are a
total of 3 ways to reach the bottom-right corner:
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down
```

Constraints:

- $1 \leq m, n \leq 100$

Claude Opus Solution

Unique Paths — Complete Solution Guide

1. Intuition

The robot can **only move down or right**. To get from the top-left `grid[0][0]` to the bottom-right `grid[m-1][n-1]`, the robot must make exactly: $m - 1$ **down** moves - $n - 1$ **right** moves

Key insight #1 (Combinatorics): The *total* number of moves is fixed at $(m-1) + (n-1) = m+n-2$. A path is uniquely defined by *which positions* in this move sequence are "down" moves (the rest are automatically "right"). So we just need to count how many ways to choose $m-1$ positions out of $m+n-2$. That's a simple combination: **$C(m+n-2, m-1)$** .

Key insight #2 (Dynamic Programming): The number of ways to reach any cell equals the sum of ways to reach the cell **above it** and the cell **to its left** (since those are the only two cells that can lead into it):

```
paths(i, j) = paths(i-1, j) + paths(i, j-1)
```

with base case: any cell in the first row or first column has exactly **1** path (you can only go straight).

2. Approach (Pseudocode)

DP Approach (most instructive)

- Create a 1D array `dp` of size `n`, initialized to 1
(this represents the first row: only one way to reach each cell)
- For each subsequent row (from row 1 to `m-1`):
For each column `j` from 1 to `n-1`:
`dp[j] = dp[j]` (paths coming from ABOVE - old value)
`+ dp[j-1]` (paths coming from LEFT - already updated)
- Return `dp[n-1]` (the bottom-right cell)

The trick: a single 1D array works because when we update `dp[j]`, the *old* `dp[j]` is the "above" value, and the *new* `dp[j-1]` (already updated this row) is the "left" value.

3. Python Solutions

Solution A: Dynamic Programming (Space-Optimized)

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[j] = number of unique paths to reach the current row's column j.
        # Initialize as the first row: every cell reachable in exactly 1 way
        # (robot can only move right along the top edge).
```

```

dp = [1] * n

# Process each remaining row. We don't need to track the actual row
# index for the values—just iterate (m-1) more times.
for _ in range(1, m):
    # Column 0 stays at 1 (first column: only one way—move straight down),
    # so we start updating from column 1.
    for j in range(1, n):
        # Before this line runs:
        #   dp[j] = paths to the cell directly ABOVE (old row's value)
        #   dp[j-1] = paths to the cell to the LEFT (already updated this row)
        # The new value sums both incoming directions.
        dp[j] = dp[j] + dp[j - 1]

# The last element holds paths to the bottom-right corner.
return dp[-1]

```

Solution B: Combinatorial Math (Optimal)

```

from math import comb

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # Total moves = (m-1) downs + (n-1) rights = m+n-2 moves.
        # Choose which of those are the (m-1) down moves: C(m+n-2, m-1).
        # comb() computes this efficiently without overflow concerns in Python.
        return comb(m + n - 2, m - 1)

```

4. Complexity Analysis

Solution A (DP)

- **Time:** $O(m \times n)$ — we fill an $m \times n$ conceptual grid, visiting each cell once.
- **Space:** $O(n)$ — only a single row of size n is kept (rolling array optimization).

Solution B (Math)

- **Time:** $O(\min(m, n))$ — `comb(a, b)` multiplies/divides roughly $\min(b, a-b)$ terms. (The README states $O(m+n)$; in practice it's bounded by the smaller of $m-1/n-1$.)
- **Space:** $O(1)$ — just a few integers (Python big-int handling aside).

Note: Solution B is strictly better in both time and space. Solution A is more valuable for *learning* DP and generalizes to harder variants (e.g., grids with obstacles — LeetCode 63).

5. Alternative Approaches

Approach	Time	Space	Notes
Combinatorics	$O(\min(m,n))$	$O(1)$	Best. Direct formula $C(m+n-2, m-1)$.
1D DP (rolling array)	$O(m \cdot n)$	$O(n)$	Shown above. Great teaching solution.
2D DP	$O(m \cdot n)$	$O(m \cdot n)$	Full grid <code>dp[i][j] = dp[i-1][j] + dp[i][j-1]</code> . Easiest to visualize.
Recursion + Memoization	$O(m \cdot n)$	$O(m \cdot n)$	Top-down: <code>f(i,j) = f(i-1,j) + f(i,j-1)</code> , cache results.
Plain Recursion	$O(2^{(m+n)})$	$O(m+n)$	Exponential—too slow, only for understanding.

2D DP version (for clarity, if you're new to DP):

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[i][j] = number of unique paths to reach cell (i, j)
        dp = [[1] * n for _ in range(m)] # first row & col are all 1

        for i in range(1, m):
            for j in range(1, n):
                # Sum of paths from above and from the left
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[m - 1][n - 1]

```

Key Pattern Takeaway

This is the classic "**grid path counting**" DP pattern. Whenever you can only move in restricted directions toward a target and want to *count paths*, the recurrence is almost always "**sum of ways from all cells that can step into the current cell.**" Recognizing this lets you instantly solve related problems like *Unique Paths II* (obstacles), *Minimum Path Sum*, and *Dungeon Game*.

Codex Solution

1. Intuition

To go from the top-left to the bottom-right:

- The robot must move **down** exactly $m - 1$ times.
- The robot must move **right** exactly $n - 1$ times.

So every valid path is just an ordering of these moves.

For example, if $m = 3$, $n = 7$:

- Down moves = 2
- Right moves = 6
- Total moves = 8

We need to choose which 2 of the 8 positions are down moves.

So the answer is:

$\binom{m+n-2}{m-1}$

or equivalently:

$\binom{m+n-2}{n-1}$

This is a classic **combinatorics / counting paths** problem.

2. Approach

We compute:

$\binom{m+n-2}{\min(m-1, n-1)}$

We use the smaller of $m - 1$ and $n - 1$ to reduce the number of multiplications.

Pseudocode

```
total_moves = m + n - 2
moves_to_choose = min(m - 1, n - 1)

answer = 1

for i from 1 to moves_to_choose:
    answer = answer * (total_moves - moves_to_choose + i)
    answer = answer / i

return answer
```

3. Python Solution

class Solution:

```
def uniquePaths(self, m: int, n: int) -> int:
    # To reach the bottom-right corner:
    # We need exactly (m - 1) down moves
    # and exactly (n - 1) right moves.
    #
    # Total moves = (m - 1) + (n - 1) = m + n - 2.
    total_moves = m + n - 2

    # We can choose either:
    # - positions of the down moves, or
    # - positions of the right moves.
```

```

#
# Choose the smaller one to minimize computation.
choose = min(m - 1, n - 1)

# We compute C(total_moves, choose) using the formula:
#
#  $C(N, k) = N! / (k! * (N - k)!)$ 
#
# Instead of computing factorials directly, we multiply and divide
# step by step to avoid large intermediate values.
paths = 1

for i in range(1, choose + 1):
    # This builds the binomial coefficient incrementally.
    #
    # Example:
    #  $C(8, 2) = (7 * 8) / (1 * 2)$ 
    paths = paths * (total_moves - choose + i) // i

return paths

```

4. Complexity Analysis

Let:

$k = \min(m - 1, n - 1)$

Time Complexity

$[O(k)]$

We run one loop from 1 to k.

Since $k \leq \min(m, n)$, this is:

$[O(\min(m, n))]$

Space Complexity

$[O(1)]$

We only use a few integer variables.

5. Alternative Approaches

1. Dynamic Programming

We can define:

```
dp[i][j] = number of ways to reach cell (i, j)
```

Since the robot can only come from the top or left:

```
dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
```

This can be optimized to a 1D array.

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[j] represents the number of ways to reach the current row's column j.
        dp = [1] * n

        for i in range(1, m):
            for j in range(1, n):
                dp[j] = dp[j] + dp[j - 1]

        return dp[-1]

```

Complexity:

- Time: $O(m * n)$
 - Space: $O(n)$
-

2. Built-in Combinatorics

Python has `math.comb`, which directly computes combinations.

```
from math import comb

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        return comb(m + n - 2, m - 1)
```

This is the shortest solution, but the manual combinatorics version shows the underlying logic more clearly.